

CS-4110 HPC with GPUs

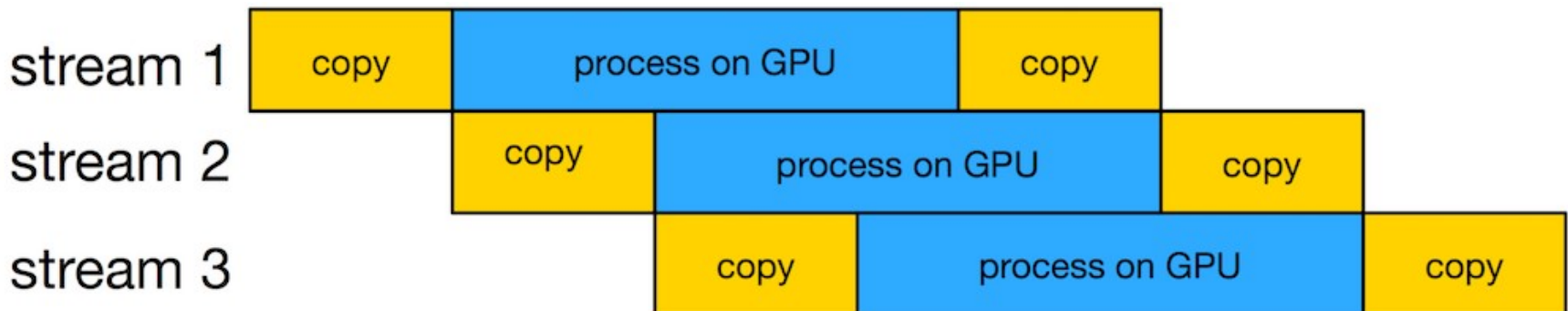
Dr Imran Ashraf
Associate Professor
CS, NUCES-FAST, Islamabad
C-205-E

CPU-GPU communication optimizations

- CPU-GPU data transfer is expensive
- Avoid un-necessary moves
- Minimize necessary moves
- Batch small transfers to one large transfer
- Use pinned memory
- **Overlap data transfer with kernel execution**

Streams

- A sequence of operations that execute in issue-order on the GPU
 - CUDA operations in different streams may run concurrently
 - CUDA operations from different streams may be interleaved



Streams in action

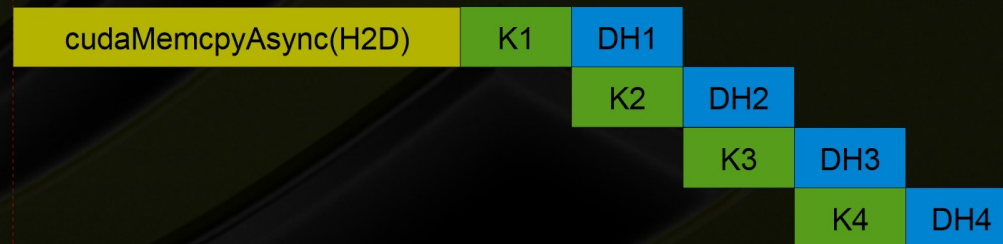
Serial



time

Concurrent – overlap kernel and D2H copy

streams



1.33x
performance
improvement

time

Example: Synchronous

```
cudaMalloc ( &dev1, size ) ;  
double* host1 = (double*) malloc ( &host1, size ) ;  
...
```

```
cudaMemcpy ( dev1, host1, size, H2D ) ;  
kernel2 <<< grid, block, 0 >>> ( ..., dev2, ... ) ;  
kernel3 <<< grid, block, 0 >>> ( ..., dev3, ... ) ;  
cudaMemcpy ( host4, dev4, size, D2H ) ;  
...
```



**completely
synchronous**

- **All CUDA operations in the default stream are synchronous**

Asynchronous without streams

```
cudaMalloc ( &dev1, size ) ;  
double* host1 = (double*) malloc ( &host1, size ) ;  
...
```

```
cudaMemcpy ( dev1, host1, size, H2D ) ;  
kernel2 <<< grid, block >>> ( ..., dev2, ... ) ;  
some_CPU_method () ;  
kernel3 <<< grid, block >>> ( ..., dev3, ... ) ;  
cudaMemcpy ( host4, dev4, size, D2H ) ;  
...
```



**potentially
overlapped**

- GPU kernels are asynchronous with host by default**

Asynchronous with streams

```
cudaStream_t stream1, stream2, stream3, stream4 ;  
cudaStreamCreate ( &stream1 ) ;
```

```
...
```

```
cudaMalloc ( &dev1, size ) ;  
cudaMallocHost ( &host1, size ) ;
```

```
...
```

```
cudaMemcpyAsync ( dev1, host1, size, H2D, stream1 ) ;  
kernel2 <<< grid, block, 0, stream2 >>> ( ..., dev2, ... ) ;  
kernel3 <<< grid, block, 0, stream3 >>> ( ..., dev3, ... ) ;  
cudaMemcpyAsync ( host4, dev4, size, D2H, stream4 ) ;  
some_CPU_method () ;
```

```
...
```

// pinned memory required on host

} .

**potentially
overlapped**

- **Fully asynchronous / concurrent**
- **Data used by concurrent operations should be independent**

Array increment: 1/**No** CUDA Streams

- Vector sum example, $\text{Arr} = \text{Arr} + 1$



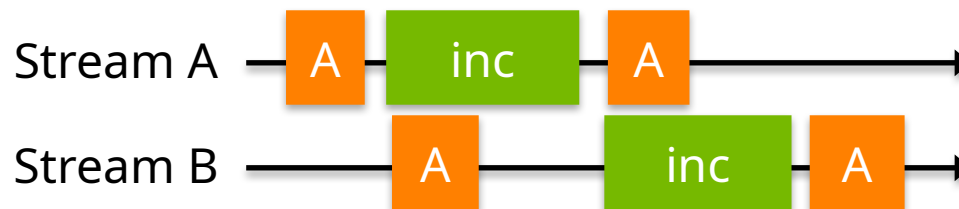
How will this execution look like
when using CUDA Streams?

Array increment: 2 CUDA Streams

- Vector sum example, $Arr = Arr + 1$



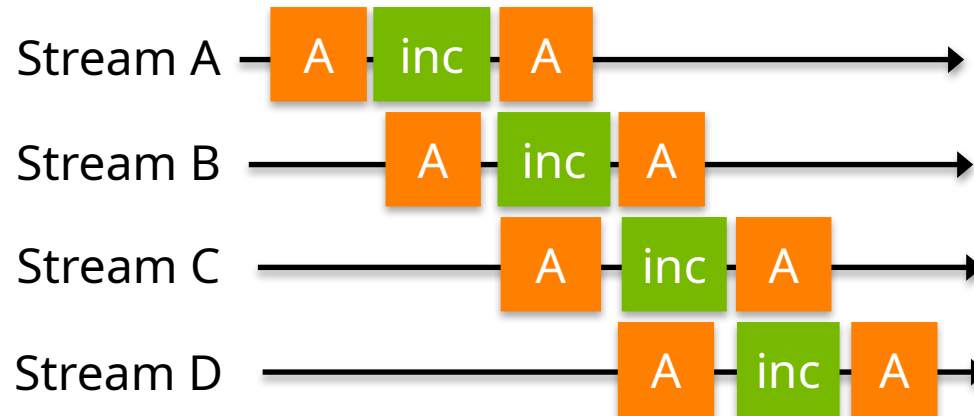
- Partition the vectors and use CUDA streams to overlap copy and compute



Quantitatively, what will be the improvement by using 2 streams?

Array increment: 4 CUDA Streams

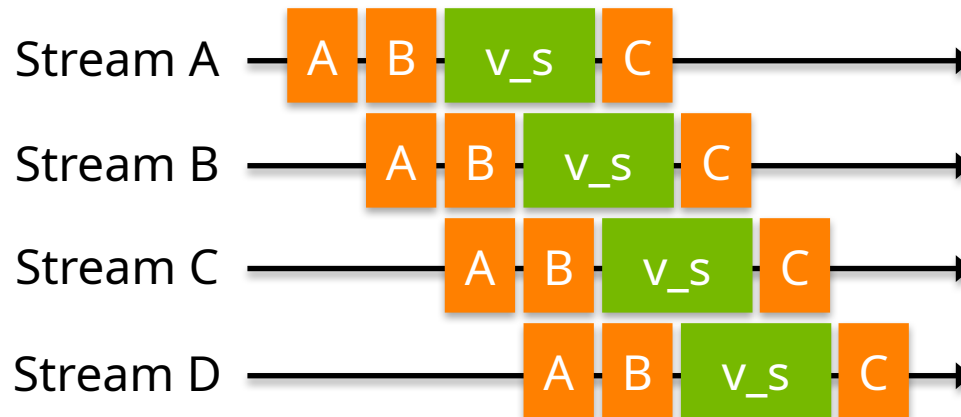
- Partition the vectors and use CUDA streams to overlap copy and compute



Quantitatively, what will be the improvement by using 4 streams?

Vector Add: CUDA Streams

- Partition the vectors and use CUDA streams to overlap copy and compute



Vector sum Example: CUDA Streams

- How can this be implemented in code?

```
for (int i = 0; i < nstreams; i++) {
    int offset = i * eles_per_stream;
    cudaMemcpyAsync(&d_A[offset], &h_A[offset], eles_per_stream *
        sizeof(int), cudaMemcpyHostToDevice, streams[i]);
    cudaMemcpyAsync(&d_B[offset], &h_B[offset], eles_per_stream *
        sizeof(int), cudaMemcpyHostToDevice, streams[i]);
    .....
    vector_sum<<<..., streams[i]>>>(d_A + offset,
        d_B + offset, d_C + offset);
    cudaMemcpyAsync(&h_C[offset], &d_C[offset], eles_per_stream *
        sizeof(int), cudaMemcpyDeviceToHost, streams[i]);
}

for (int i = 0; i < nstreams; i++)
    cudaStreamSynchronize(streams[i]);
```

Performance of Tiled DGEMM

- **CPU** (4core Westmere x5670 @2.93 GHz, MKL)

- **43 Gflops**

- **GPU** (C2070)

- **Serial : 125 Gflops (2.9x)**

- **2-way : 177 Gflops (4.1x)**

- **3-way : 262 Gflops (6.1x)**

- **GPU + CPU**

- **4-way con.: 282 Gflops (6.6x)**

- **Up to 330 Gflops for larger rank**

- **Obtain maximum performance by leveraging concurrency**

- **All communication hidden – effectively removes device memory size limitation**

DGEMM: $m=n=8192$, $k=288$

Nvidia Visual Profiler (nvvp)

default stream

stream 1

stream 2

stream 3

stream 4

CPU

